

FinSync

Automated Payment Integration Pipeline

Technical Interview Deep-Dive & Project Reference

This document explains the FinSync integration microservice end to end: its architecture, the reasoning behind every design decision, the data flow of a single event, and a simulated deep-dive interview covering the questions an integration-engineering panel is most likely to ask.

Author: Mayank Karidhi | Repo: github.com/MayankKaridhi/finsync

Stack: Node.js - Express - Zod - Axios - Winston - Helmet

1. What FinSync Is (30-Second Pitch)

FinSync is an event-driven backend microservice that sits between a payment provider and a downstream accounting system. It exposes a secured webhook endpoint that receives real-time transaction events, verifies and validates them, transforms the JSON into the accounting system's schema, and reliably forwards it - retrying transient failures and dead-lettering anything that cannot be delivered, so no financial event is ever silently lost.

The one-sentence version for an interviewer: "It's a resilient webhook gateway that does receive -> verify -> validate -> transform -> forward, built around the three rules every integration must follow: acknowledge fast, never trust the payload, and assume the downstream will fail."

The three guiding principles

- Acknowledge fast: validate, then return 202 Accepted immediately. The heavy transform-and-forward work happens after the ACK so a slow downstream never causes the provider to time out and re-deliver.
- Never trust the payload: every request is HMAC-verified, then validated against a strict Zod schema at the boundary. Bad input gets a 400 and never reaches business logic.
- Assume the downstream fails: the outbound connector retries transient faults with exponential backoff, refuses to retry permanent 4xx errors, and dead-letters exhausted events for replay.

2. Architecture & Request Lifecycle

A single webhook travels through seven stages inside the gateway. Each stage has exactly one job, which keeps every module small and testable.

```
[Payment source] --signed webhook POST--> FINSYNC GATEWAY

1. Helmet security headers + correlation-id tag
2. HMAC-SHA256 signature verify ..... fail -> 401
3. Zod schema validation ..... fail -> 400
4. Acknowledge fast ..... 202 Accepted
   | (processing continues after ACK)
5. Transform (cents->decimal, status/dir map)
6. Connector: Axios POST to accounting API
   |-- retry 5xx/429 w/ backoff+jitter
   +-- exhausted -> dead-letter (NDJSON)
7. Winston structured logs at every step

--> [Mock Accounting System] (idempotent ledger)
```

Layered module layout (and why)

- config/ - one validated source of truth for env vars; fails loudly at boot.
- schemas/ - the Zod trust boundary; nothing downstream assumes anything it does not guarantee.
- middleware/ - signature verification and the central error handler.
- services/ - the domain core: transformer (pure), connector (retry), deadLetter (durability), pipeline (orchestration).
- controllers/ + routes/ - thin HTTP layer; validate, ACK, delegate.
- app.js / server.js - app factory (dependency-injectable for tests) and the process entry point with graceful shutdown.

Why this separation matters in an interview

The controller's job ends at the ACK; the IntegrationPipeline does the real work. That seam is deliberate: in a scaled deployment the controller would push the event onto a queue (SQS/Kafka) and a worker pool would run the exact same pipeline class. Nothing in the pipeline knows or cares whether it was triggered by an HTTP handler or a queue consumer.

3. The Transformation - Where Integration Actually Happens

Two systems that were never designed to talk to each other are reconciled field by field in one pure function, `toJournalEntry()`. Pure means: same input always yields the same output, no I/O, no hidden state - which is why it is trivially unit-testable.

Inbound (payment webhook) -> Outbound (journal entry)

```
event_id          -> idempotency_key    (dedupe at-least-once delivery)
data.amount=1999  -> entry.amount="19.99" (cents int -> decimal STRING)
data.currency=usd -> entry.currency="USD" (normalised upper-case)
data.status       -> entry.status="POSTED" (ledger vocabulary)
event_type        -> entry.direction="CREDIT"/"DEBIT"
data.customer.*   -> counterparty.*
```

The single most important detail here: money is carried as an INTEGER number of cents end to end, and only rendered to a "19.99" STRING at the very edge. Floating point is never used for money because $0.1 + 0.2 \neq 0.3$ in IEEE-754 - using a float risks losing a cent, which in a financial system is a defect, not a rounding curiosity.

4. Simulated Technical Deep-Dive (Q&A)

These are the questions an integration-engineering panel is most likely to ask about this project, with the answers the code actually supports.

Q: Walk me through what happens when a webhook hits your service.

A:

A request first passes Helmet and gets tagged with a correlation ID. Then the signature middleware recomputes an HMAC-SHA256 over the raw request body using the shared secret and compares it to the X-FinSync-Signature header - a mismatch is a 401. Next the controller validates the parsed body against a Zod schema; a malformed payload is a 400 with field-level detail.

If both pass, I respond 202 Accepted immediately with the correlation ID. Only after that do I run the pipeline: transform the payload into a journal entry, then forward it to the accounting API via Axios with retry/backoff. If every retry is exhausted, the event is written to a dead-letter log instead of being dropped.

Q: Why acknowledge with 202 before you've actually forwarded the data?

A:

Webhook senders treat a slow or missing response as a failure and re-deliver, often aggressively. If I blocked the HTTP response on a slow accounting API, a downstream hiccup would trigger a retry storm from the provider and amplify the problem. Acknowledging fast decouples my availability from the downstream's. 202 (not 200) is deliberate - it means 'accepted for processing', which is exactly the contract: I've durably taken responsibility for the event, not necessarily finished it.

Q: How do you make sure you don't lose an event if the accounting system is down?

A:

Two layers. First, the connector retries transient failures - network errors, 5xx, and 429 - with exponential backoff and jitter, up to a configured max. Second, if all retries are exhausted, the event is written to a dead-letter store as NDJSON containing both the original payload and the transformed body, plus the failure reason and correlation ID. That record can be inspected and replayed. In production the file-based DLQ becomes an SQS/Kafka dead-letter queue, but the semantics - nothing is ever dropped - are identical.

Q: Why do you retry a 503 but not a 400?

A:

A 503 or a network timeout is transient: the downstream might recover a moment later, so retrying is likely to succeed. A 4xx like 400 (bad request) or 409 (duplicate) is a permanent, deterministic outcome - the same request will fail the same way forever. Retrying it just hammers the downstream and delays the inevitable dead-letter. So `isRetryable()` returns true only for no-response errors, 429, and 5xx.

Q: What is the jitter for in your backoff?**A:**

Without jitter, if many events fail at the same instant (say the downstream briefly went down), they'd all retry at exactly $\text{base} \cdot 2^n$ and hit the recovering service in synchronized waves - the thundering-herd problem. Adding a small randomized offset spreads the retries out so the downstream can actually recover instead of being knocked over again.

Q: Why validate with a schema instead of just reading the fields you need?**A:**

The schema is the trust boundary. A public webhook endpoint is reachable by anyone, and upstream systems change payloads without telling you. Zod lets me declare the exact shape once - types, enums, formats, an integer-cents constraint on amount - and reject anything that doesn't conform before it reaches business logic. It also strips unknown keys, so a sender can't smuggle extra fields into the transform. And because Zod coerces (e.g. lower-casing currency), the rest of the code works with clean, normalized data.

Q: How would you handle the same webhook being delivered twice?**A:**

Webhook delivery is at-least-once, so duplicates are expected, not exceptional. I map `event_id` to an `idempotency_key` on the journal entry, and the downstream uses it to dedupe - a repeated key returns the existing entry with `duplicate:true` rather than creating a second ledger row. My end-to-end test proves this: replaying the same event leaves the ledger count at one.

Q: How do you verify the webhook actually came from the payment provider?**A:**

HMAC-SHA256 over the raw body with a shared secret. Two subtle but important details: I hash the raw bytes captured by the body-parser's verify hook, not the re-serialized object, because re-serializing can reorder keys or change whitespace and break the comparison. And I compare with `crypto.timingSafeEqual`, not `===`, to avoid leaking information through a timing side-channel.

Q: How is this observable in production - how would you debug one failed sync?**A:**

Every request gets a correlation ID at ingress, echoed in the `X-Correlation-Id` response header and attached to every Winston log line across validation, transform, each retry attempt, and the dead-letter write. Logs are structured JSON, so in an aggregator I can filter by that one ID and see the entire lifecycle of a single event end to end - which is the first thing you want when a specific transaction didn't sync.

Q: How did you test something this asynchronous without flaky tests?**A:**

Dependency injection. The connector takes an injectable Axios client and sleep function, so retry tests script exact responses (503, 503, 201) and assert the attempt count with zero real network or real delays. The pipeline takes an injectable connector and dead-letter store. On top of the 25 unit/HTTP tests, an end-to-end runner boots the real gateway and the real mock downstream, injects two 503s, and asserts the entry still lands, is idempotent on replay, and that a bad payload is rejected.

Q: What would you change to scale this to millions of events a day?**A:**

The controller would enqueue the validated event instead of processing inline, and a horizontally-scaled worker pool would consume the queue running the same IntegrationPipeline. The dead-letter file becomes a managed DLQ. I'd add a persistent idempotency store (Redis/DB) shared across workers, and metrics (Prometheus) on sync latency, retry counts, and DLQ depth. The code is already structured for this - the pipeline is transport-agnostic.

5. Resume Bullets & Talking Points

ATS-friendly bullets this project genuinely supports:

- Built an event-driven payment integration microservice in Node.js and Express.js exposing a secured REST API webhook listener that ingested real-time transaction JSON payloads and acknowledged events with 202 to prevent upstream retry storms.
- Engineered a data-transformation layer that parsed, schema-validated (Zod), and restructured inbound JSON into a normalized accounting schema, then forwarded records via Axios in an automated end-to-end data-sync pipeline.
- Implemented fault-tolerant error handling with retry/exponential-backoff, structured logging (Winston), and a dead-letter queue to track data-sync validation failures and guarantee zero event loss across the webhook-to-API pipeline.

Key terms you can now defend

Webhooks and at-least-once delivery, idempotency keys, HMAC signature verification, schema validation at the trust boundary, data transformation, exponential backoff with jitter, retryable vs permanent errors, dead-letter queues, correlation IDs and structured logging, dependency injection for testing, and the accept-fast/process-after-ACK pattern.

An honest note for your interviews

Be ready to actually run this and explain any line - that is what makes it yours. Run `npm test` (25 passing) and `npm run e2e` (full pipeline) so you have seen the output first-hand. If asked what you would improve, good candid answers are: a persistent shared idempotency store, a real message queue instead of inline processing, and metrics/alerting on dead-letter depth. Knowing the current limits is a strength, not a weakness.